

Assignment 6: Orbital Perturbations

1 Introduction

In this assignment the student will learn how to propagate orbits that include perturbations. This will be done by extending `orbit.lib.py` with functionality for orbit propagation with perturbations.

After completing this assignment the student will learn:

- How to implement orbit propagators that take perturbations into consideration.

2 Preliminaries

2.1 Earth Model

We will model the Earth as a spheroid with parameters based on WGS-84

- Equatorial radius: $R_E = 6378.137$ km
- Flattening: $f = \frac{1}{298.257223563}$
- Second Zonal Harmonic: $J_2 = 0.001082629821313$
- Rotational velocity: $\omega_E = 7.292115 \cdot 10^{-5}$ rad/s
- Gravitational parameter: $\mu = 398600.4418$ km³/s²

2.2 PKepler propagator

Over the decades many different algorithms have been developed for propagating orbital parameters in the prescence of perturbations. A simple algorithm to propagate TLE parameters that accounts for Earth's oblateness and atmospheric drag is given in [1] with the name 'PKepler'. Although one should use SGP4 when propagating TLE parameters this algorithm has a decent tradeoff between ease of implementation, speed and accuracy. The 'PKepler' propagator can be briefly summarized as

Initialization: Provided initial parameters: $a, e, M_e, \Omega, i, \omega, \dot{n}, \ddot{n}$.

Propagation: Given a time step Δt first compute

$$p = a_o (1 - e_o^2)$$

$$n = \sqrt{\frac{\mu}{a_o^3}}$$

Where $_o$ denotes the orbital parameter before the update. Update orbital parameters $a, e, \Omega, \omega, M_e$

$$a = a_o - \frac{2a_o}{3n} \cdot \Delta t$$

$$e = e_o - \frac{2(1 - e_o)}{3n} \dot{n} \cdot \Delta t$$

$$\Omega = \Omega_o - \frac{3nR_E^2 J_2}{2p^2} \cos(i) \cdot \Delta t$$

$$\omega = \omega_o + \frac{3nR_E^2 J_2}{4p^2} (4 - 5 \sin(i)) \cdot \Delta t$$

$$M_e = M_{eo} + n \cdot \Delta t + \frac{1}{2} \dot{n} \cdot \Delta t^2 + \frac{1}{6} \ddot{n} \cdot \Delta t^3$$

if $M_e > 2 \cdot \pi \rightarrow M_e = M_e - 2 \cdot \pi$.

With this algorithm we can propagate satellites in 'real-time'. By getting the current time and date using the 'datetime' package

```

1 import datetime as dt
2 tnow = dt.datetime.now()
3 JD_now = ol.datetime_to_julian_date(tnow.year, tnow.month,
    tnow.day, tnow.hour, tnow.minute, tnow.second)

```

we can convert it to a julian date (If you haven't already made this function then refer to Assignment 2 section 2.9.2). Then by subtracting it from the Epoch julian date and multiplying with $24 \cdot 3600$ we get Δt from the Epoch to now

```

1 JD_E = ol.epoch_to_julian_data(epoch)
2 delta_t = (24*3600)*(JD_E-JD_now)

```

We can therefore first initialize the orbit, then propagate it until the current time using 'delta_t' and then we can simulate the satellite 't_e' into the future. Note that you would initialize the Earth's rotation using 'JD_now'.

3 Two-Line Elements

The 'PKepler' propagator requires the parameters $\dot{n}$ and $\ddot{n}$ which is also contained within the TLE format. From Assignment 2 we defined the parameters HST

```

1 20580U 90037B 17257.12407589 .00000765 00000-0 36248-4 0 9997
2 20580 28.4693 139.8141 0002810 52.8630 75.3801 15.08848241303732

```

Row 1, Field 3: Epoch: NN|NNN.NNNNNNNN $\rightarrow$ Year|DayinYear.FractionofDay

Row 2, Field 2: Inclination i : [Degrees]

Row 2, Field 3: RAAN Ω : [Degrees]

Row 2, Field 4: Eccentricity e : NNNNNNN $\rightarrow$ 0.NNNNNNN

Row 2, Field 5: Argument of perigee ω : [Degrees]

Row 2, Field 6: Mean anomaly M_e : [Degrees]

Row 2, Field 7: NN.NNNNNNNN|NNNNN|N $\rightarrow$ Revs per day|Revs|Checksum

Where the mean motion is obtained from 'Revs per day' directly using

$$n = \frac{2\pi \cdot \text{Revs per day}}{24 \cdot 3600}$$

In addition we can also get $\dot{n}$, $\ddot{n}$ and Bstar from

Row 1, Field 4: First derivative of Mean Motion $\dot{n}$: S.NNNNNNNN $\rightarrow \pm 0.NNNNNNNN$

Row 1, Field 5: Second derivative of Mean Motion $\ddot{n}$: SNNNNN-E $\rightarrow \pm 0.NNNNN \cdot 10^{-E}$

Row 1, Field 6: Bstar Coefficient: SNNNNNSE $\rightarrow \pm 0.NNNNN \cdot 10^{\pm E}$

where 'S' can be either '+' or '-'. The parameters in the TLE are not in a appropriate format for the PKepler propagator such that the following transformations needs to be done

$$\dot{n} = \frac{4\pi}{(24 \cdot 3600)^2} \dot{\hat{n}}$$

$$\ddot{n} = \frac{12\pi}{(24 \cdot 3600)^3} \ddot{\hat{n}}$$

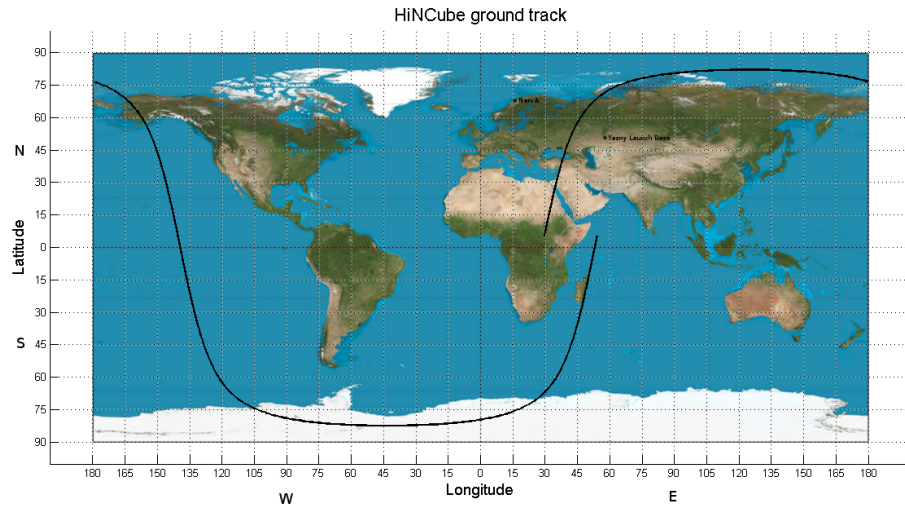


Figure 1: Ground Track of HiNCube showing its first orbit after launch.

Bstar is related to the ballistic coefficient, but it is not used in the PKepler propagator. The appendix lists a version of 'read_TLE_file' that also returns $\dot{\vec{n}}$, $\ddot{\vec{n}}$ and 'Bstar'.

3.1 Ground Tracks

Ground tracks are a useful visualisation of the current position of the satellite as shown in Figure 1. To be able to plot a ground track it is necessary to know the geodetic coordinates of the satellite. An algorithm to convert from ECEF coordinates to geodetic coordinates is given in the Appendix.

4 Assignment 6: PKepler propagator

Assignment 6

1. Extend Orbit_lib.py with the convenience functions

$$\phi, \lambda, r = \text{geocentric_from_xyz}(\mathbf{r}^E)$$

$$\phi, \lambda', h = \text{geodetic_from_xyz}(\mathbf{r}^E)$$

$$\mathbf{r}^E = \text{xyz_from_geodetic}(\phi, \lambda', h)$$

$$\mathbf{r}^E = \text{xyz_from_geocentric}(\phi, \lambda, r)$$

2. Extend orbit_lib.py with a orbit_pkepler propagator class.
3. Extend simutils.py with the new TLE reader.
4. Extend plotter.py to be able to plot ground tracks.
5. Simulate Assignment 5.3 using the PKepler propagator from the current time and plot the ground tracks.

References

- [1] David A. Vallado, *Fundamentals of Astrodynamics and Applications*, Microcosm Press, 4th edition, 2013.

A Read_TLE_file

```

1 def read_TLE_file(file_name, satellite_name=''):
2     def validate_entry(Name, line1, line2):
3         if not Name[0].isalpha():
4             return False
5         if not line1[0].startswith("1") or not len(line1) == 9:
6             return False
7         if not line2[0].startswith("2") or not len(line2) == 8:
8             return False
9         return True
10    tle_data = []
11    with open(file_name) as f:
12        file_contents = f.readlines()
13    if len(file_contents) < 3:
14        print("Error reading file\nRequired format is:\n
nAAAAAAAAAAAAAAAAAAAAAAAAA\n1 NNNNNU NNNNNAAA NNNNN.
NNNNNNNN +.NNNNNNNN +NNNNN-N +NNNNN-N N NNNNN\n2 NNNNN
NNN.NNNN NNN.NNNN NNNNNNN NNN.NNNN NNN.NNNN NN.
NNNNNNNNNNNNNN\nfor each entry")
15    return tle_data
16    for i in range(0, len(file_contents), 3):
17        if (satellite_name in file_contents[i]):
18            Name = file_contents[i].strip()
19            line1 = file_contents[i+1].strip().split()
20            line2 = file_contents[i+2].strip().split()
21            if validate_entry(Name, line1, line2):
22                _, _, _, epoch, sdn, sddn, sbstar, _, _ = line1
23                _, _, si, s0, secc, sw, sM, srev = line2
24                epoch = float(epoch)
25                dn = float(sdn)
26                s1, s2 = sddn.split(sddn[-2])
27                if s1[0].isdigit():
28                    ddn = float("."+s1+"e"+sddn[-2]+s2)
29                else:
30                    ddn = float(s1[0]+"."+s1[1:]+"e"+sddn[-2]+s2)
31                s1, s2 = sbstar.split(sbstar[-2])
32                if s1[0].isdigit():
33                    bstar = float("."+s1+"e"+sbstar[-2]+s2)
34                else:
35                    bstar = float(s1[0]+"."+s1[1:]+"e"+sbstar[-2]+s2)
36            )
37            e = float("."+secc)
38            rev = float(srev[:-6])
39            Me = float(sM)
40            i = float(si)
41            O = float(s0)
42            w = float(sw)
43            tle_data.append((Name, epoch, e, rev, Me, i, O, w, dn, ddn,
bstar))
44        else:
45            print("Error reading entry:\n{n}-{s}-{b}".format(
file_contents[i], file_contents[i+1], file_contents[i+2]))
break

```

46 `return tle_data`

Listing 1: Example of a function that reads TLE data from a textfile.

```

1  def plot_ground_track(file_path):
2      phi, lam, h = np.loadtxt(file_path, unpack=True)
3      r_g = np.vstack((phi, lam, 1e3*h)).T
4      r_idx = np.nonzero(np.abs(np.diff(r_g[:,0]))-np.pi/2 >
5                          0)[0]
6      fig, ax = plt.subplots()
7      with Image.open("earth_grid.jpg") as im:
8          im = ax.imshow(im, extent=[-180, 180, -90, 90])
9          start = 0
10         for idx in r_idx:
11             ax.plot(r_g[start:idx,0]/np.pi*180, r_g[start:idx
12                 ,1]/np.pi*180, linewidth=2, color='black')
13             start = idx+1
14             ax.plot(r_g[start:,0]/np.pi*180, r_g[start:,1]/np.pi
15                 *180, linewidth=2, color='black')
16             ax.grid(True)
17             ax.set_yticks(np.arange(-90,90+15,15))
18             ax.set_xticks(np.arange(-180,180+15,15))
19             ax.set_ylim(-90, 90)
20             ax.set_xlim(-180, 180)
21             ax.set_xlabel('Longitude', ylabel='Latitude',
22                 title='Ground track')
23             plt.show()

```

Listing 2: Example of a function that plots the ground track of a satellite given geodetic coordinates.

A.1 Algorithm 1: ECEF to Geocentric**Input:** $\mathbf{r}^E = [x \ y \ z]^\top$ **Output:** ϕ, λ, h **Step 1:** Convert from cartesian to geocentric longitude and latitude

$$r = \|\mathbf{r}^E\|$$

$$\phi = \arctan 2(y, x)$$

$$\lambda = \arctan 2\left(z, \sqrt{x^2 + y^2}\right)$$

A.2 Algorithm 2: Geocentric to ECEF

Input: ϕ, λ, h

Output: $\mathbf{r}^E = [x \ y \ z]^\top$

Step 1: Convert from geocentric longitude and latitude to xyz

$$\mathbf{r}^E = \begin{bmatrix} r \cos \phi \cos \lambda \\ r \sin \phi \cos \lambda \\ r \sin \lambda \end{bmatrix}$$

A.3 Algorithm 3: ECEF to Geodetic

Input: $\mathbf{r}^E = [x \ y \ z]^\top$

Output: ϕ', λ', h

Step 1: Convert from cartesian to geocentric longitude and latitude

$$\begin{aligned} r &= \|\mathbf{r}^E\| \\ \phi &= \arctan 2(y, x) \\ \lambda &= \arctan 2\left(z, \sqrt{x^2 + y^2}\right) \end{aligned}$$

Step 2: Initialize geodetic latitude

$$\lambda'_0 = \lambda$$

Step 3: Iterate until λ'_n converges $|\lambda'_{n+1} - \lambda'_n| < \epsilon$, for some $0 < \epsilon < 1$

$$\begin{aligned} N_n &= \frac{R_E}{\sqrt{1 - (2f - f^2) (\sin \lambda'_n)^2}} \\ \lambda'_{n+1} &= \arctan 2\left(z + N_n \cdot (2f - f^2) \cdot \sin \lambda'_n, \sqrt{x^2 + y^2}\right) \end{aligned}$$

Step 4: Compute geodetic coordinates

$$\begin{aligned} h &= \frac{\sqrt{x^2 + y^2}}{\cos \lambda'_n} - N_n \\ \phi' &= \phi \\ \lambda' &= \lambda'_n \end{aligned}$$

A.4 Algorithm 3: Geodetic to ECEF

Input: ϕ', λ', h

Output: $\mathbf{r}^E = [x \ y \ z]^\top$

Step 1: Compute N

$$N = \frac{R_E}{\sqrt{1 - (2f - f^2)(\sin \lambda')^2}}$$

Step 2: Compute ECEF coordinates

$$\mathbf{r}^E = \begin{bmatrix} (N + h) \cos \phi \cos \lambda' \\ (N + h) \sin \phi \cos \lambda' \\ (N(1 - f)^2 + h) \sin \lambda' \end{bmatrix}$$